```
#!/usr/bin/env python3
"""
Weather CLI Tool
A beginner-friendly Python project
demonstrating:
- Making HTTP requests (the 'requests'
library)
- Working with a free public API (no key
required)
- Parsing JSON responses
- Basic error handling
- argparse for command-line arguments

Setup:
    pip install requests

Usage:
    python weather.py London
    python weather.py "New York"
    python weather.py Tokyo --units
imperial
"""

import argparse
import sys

try:
    import requests
except ImportError:
    print("This program needs the
```

```
'requests' library.")
    print("Install it with: pip install
requests")
    sys.exit(1)

# Open-Meteo is a free weather API that
needs no API key or signup.
GEOCODE_URL = "https://geocoding-api.open-
meteo.com/v1/search"
WEATHER_URL = "https://api.open-
meteo.com/v1/forecast"

WEATHER_CODES = {
    0: "Clear sky",
    1: "Mainly clear",
    2: "Partly cloudy",
    3: "Overcast",
    45: "Fog",
    48: "Depositing rime fog",
    51: "Light drizzle",
    53: "Moderate drizzle",
    55: "Dense drizzle",
    61: "Slight rain",
    63: "Moderate rain",
    65: "Heavy rain",
    71: "Slight snow",
    73: "Moderate snow",
    75: "Heavy snow",
    80: "Rain showers",
    95: "Thunderstorm",
    99: "Thunderstorm with heavy hail",
}
```

```
def get_coordinates(city):
    """Look up latitude/longitude for a
city name."""
    try:
        response =
requests.get(GEOCODE_URL, params={"name":
city, "count": 1}, timeout=10)
        response.raise_for_status()
    except
requests.exceptions.RequestException as e:
        print(f"Error reaching geocoding
service: {e}")
        sys.exit(1)

    data = response.json()
    results = data.get("results")
    if not results:
        print(f"Could not find a city named
'{city}'. Check the spelling and try
again.")
        sys.exit(1)

    place = results[0]
    return place["latitude"],
place["longitude"], place["name"],
place.get("country", "")


def get_weather(lat, lon, units):
    temp_unit = "fahrenheit" if units ==
"imperial" else "celsius"
    wind_unit = "mph" if units ==
"imperial" else "kmh"
```

```
    params = {
        "latitude": lat,
        "longitude": lon,
        "current":
"temperature_2m,relative_humidity_2m,weathe
r_code,wind_speed_10m",
        "temperature_unit": temp_unit,
        "wind_speed_unit": wind_unit,
    }

    try:
        response =
requests.get(WEATHER_URL, params=params,
timeout=10)
        response.raise_for_status()
    except
requests.exceptions.RequestException as e:
        print(f"Error reaching weather
service: {e}")
        sys.exit(1)

    return response.json().get("current",
{})


def display_weather(city, country, current,
units):
    temp_symbol = "°F" if units ==
"imperial" else "°C"
    speed_unit = "mph" if units ==
"imperial" else "km/h"

    code = current.get("weather_code")
    condition = WEATHER_CODES.get(code,
```

```
"Unknown")

    print(f"\nWeather in {city},
{country}")
    print("-" * (14 + len(city) +
len(country)))
    print(f"Condition:   {condition}")
    print(f"Temperature:
{current.get('temperature_2m')}
{temp_symbol}")
    print(f"Humidity:
{current.get('relative_humidity_2m')}%")
    print(f"Wind speed:
{current.get('wind_speed_10m')}
{speed_unit}\n")


def main():
    parser =
argparse.ArgumentParser(description="Get
the current weather for a city.")
    parser.add_argument("city", type=str,
help="City name, e.g. London or 'New
York'")
    parser.add_argument(
        "--units",
        choices=["metric", "imperial"],
        default="metric",
        help="Units for temperature/wind
speed (default: metric)",
    )
    args = parser.parse_args()

    lat, lon, resolved_name, country =
```

```
get_coordinates(args.city)
    current = get_weather(lat, lon,
args.units)
    display_weather(resolved_name, country,
current, args.units)


if __name__ == "__main__":
    main()
```